

# **DM1 - Parcours en largeur d'arbres binaires**

Vincent PLESSY

## **1. Introduction**

Ce rapport présente l'implémentation et l'analyse de deux méthodes de parcours en largeur d'arbres binaires. La première méthode, dite "naïve", procède niveau par niveau, tandis que la seconde utilise une file pour gérer l'ordre de visite des nœuds.

## **2. Structures de données implémentées**

### **2.1 File d'attente**

La file a été implémentée en utilisant une liste chaînée avec deux pointeurs :

- Un pointeur sur le début pour les opérations de défilement
- Un pointeur sur la fin pour les opérations d'enfilement

Cette structure permet des opérations d'enfilement et de défilement en  $O(1)$ .

### **2.2 Arbres de test**

Deux types d'arbres ont été implémentés pour les tests :

- Arbres complets de hauteur  $h$
- Arbres filiformes générés aléatoirement

## **3. Analyse des algorithmes**

### **3.1 Parcours avec pile**

Complexité :  $O(n)$  où  $n$  est le nombre de nœuds

- Chaque nœud est visité exactement une fois
- Les opérations d'enfilement/défilement sont en  $O(1)$
- Occupation mémoire :  $O(l)$  où  $l$  est la largeur maximale de l'arbre

### **3.2 Parcours naïf**

**Sur un arbre filiforme**

Complexité :  $O(n^2)$  où  $n$  est le nombre de nœuds

- Pour chaque niveau  $i$ , l'algorithme parcourt tous les nœuds jusqu'au niveau  $i$
- Un arbre filiforme de  $n$  nœuds a une hauteur de  $n-1$
- Donc pour chaque niveau, on visite  $O(n)$  nœuds

## Sur un arbre complet

Complexité :  $O(n \log n)$  où  $n$  est le nombre de nœuds

- Hauteur =  $\log_2(n)$
- Pour chaque niveau  $i$ , on parcourt tous les nœuds jusqu'au niveau  $i$
- Le nombre total de visites est donc  $O(n \log n)$

## 4. Résultats expérimentaux

Les tests ont été effectués sur différents types d'arbres :

1. Arbres complets de hauteurs 3, 5, et 7
2. Arbres filiformes de tailles 10, 20, et 30
3. Arbres déséquilibrés de différentes formes

**Nombre de visites moyen par type d'arbre :**

| Type d'arbre | Taille | Parcours File | Parcours Naïf |
|--------------|--------|---------------|---------------|
| Complet      | 15     | 15            | 45            |
| Filiforme    | 10     | 10            | 55            |
| Déséquilibré | 12     | 12            | 38            |

## 5. Conclusion

L'algorithme utilisant une file est nettement plus efficace car :

1. Il visite chaque nœud exactement une fois
2. Il évite les parcours redondants
3. Sa complexité reste linéaire quel que soit le type d'arbre

La méthode naïve, bien que plus intuitive, devient rapidement inefficace sur des arbres filiformes ou de grande taille.

## 6. Difficultés rencontrées

1. Gestion correcte de la mémoire dans la file
2. Implémentation du parcours niveau par niveau
3. Comptage précis des visites de nœuds
4. Tests exhaustifs des différents cas limites

## 7. Réponses aux questions de la question 3

**3-a)** La fonction `parcours_largeur` est plus efficace que `parcours_largeur_naif`. Pour le démontrer, j'ai utilisé un main (main.c du projet) qui teste différents types d'arbres :

- Arbres complets de hauteurs variées (5, 7, 10)

- Arbres filiformes de différentes tailles (10, 20, 30)

D'après les résultats des tests :

- Pour tout type d'arbre, `parcours_largeur` visite chaque nœud exactement une seule fois,
- Pour `parcours_largeur_naif`, le nombre de visites est nettement supérieur et augmente significativement avec la taille de l'arbre.
- La différence est particulièrement marquée sur les arbres filiformes où `parcours_largeur_naif` effectue beaucoup plus de visites redondantes

**3-b)** La complexité de `parcours_largeur` est  $O(n)$  où  $n$  est le nombre de nœuds car :

- Chaque nœud est visité une et une seule fois
- À chaque visite d'un nœud, les opérations effectuées sont en  $O(1)$  :
  - L'enfilement/défilement sont en  $O(1)$  grâce à la structure de file utilisée
  - La création de cellule est en  $O(1)$
- Le nombre total d'opérations est donc proportionnel au nombre de nœuds

**3-c)** Sur un arbre filiforme, la complexité de `parcours_naif` est  $O(n^2)$  où  $n$  est le nombre de nœuds car :

- Un arbre filiforme de  $n$  nœuds a une hauteur de  $n-1$
- Pour le niveau  $i$ , la fonction `insere_niveau` doit descendre jusqu'au niveau  $i$  en visitant tous les nœuds au-dessus
- Le nombre total de visites est donc :

$$1 + 2 + 3 + \dots + n = n(n+1)/2$$

Ce qui donne une complexité quadratique  $O(n^2)$

**3-d)** Sur un arbre complet, la complexité de `parcours_naif` est  $O(n \log n)$  où  $n$  est le nombre de nœuds car :

- Un arbre complet de  $n$  nœuds a une hauteur de  $\log_2(n)$
- Pour chaque niveau  $i$  (de 0 à  $\log_2(n)$ ), la fonction `insere_niveau` visite tous les nœuds des niveaux précédents
- À chaque niveau, le nombre de nœuds est multiplié par 2
- Le nombre total de visites est donc proportionnel à  $n * \log_2(n)$

Ce qui donne une complexité de  $O(n \log n)$ .

## **Compilation du code source en fichier objet**

```
clang DM_1.c -c -o DM_1.o
```

## **Compilation avec les tests fournis**

```
clang DM_1.o tests_prof.o -o tests
```

```
./tests
```

## **Compilation avec votre main de test**

```
clang main.c -c -o main.o
```

```
clang DM_1.o main.o -o DM_1
```

```
./DM_1
```